

Assignment 6: Motion and Structure

Raghuveer Gummadi

Code: https://github.com/RagDoll95/CSC8830_Assignments/tree/feature/motion-tracking

A. Optical flow and motion tracking

The two samples are Penguin Walk, 00:30–01:00 [4], and Sand Marble Race, 02:00–02:30 [5]. Both are 30 seconds long and resized to 640×360 pixels, retaining 25 and 50 frames/second, respectively. The program detects corners and applies pyramidal Lucas–Kanade to successive frames [1]. Yellow arrows visualize pixel displacement. Corners are selected again each frame, so the video shows sparse optical flow, not persistent identities for individual animals or marbles.

Derivation from brightness constancy

Assume the same scene point keeps its brightness and moves by (d_x, d_y) between frames I_0 and I_1 . Then $I_1(x + d_x, y + d_y) = I_0(x, y)$. A first-order Taylor expansion gives

$$I_x d_x + I_y d_y + I_t \simeq 0, \quad I_t = I_1 - I_0.$$

Here time is measured in frames; for velocities in seconds divide displacement by Δt . One equation cannot determine two unknowns (the aperture problem). Assume a single displacement throughout a small textured window Ω and minimize

$$E(d) = \sum_{\Omega} (I_x d_x + I_y d_y + I_t)^2 = \|Ad - b\|^2, \quad A_k = [I_x(k), I_y(k)], \quad b_k = -I_t(k).$$

Setting $\nabla E = 2A^T(Ad - b) = 0$ yields

$$\underbrace{\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}}_{A^T A} \begin{bmatrix} d_x \\ d_y \end{bmatrix} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{A^T b}.$$

Two independent gradient directions are needed. A corner is useful; a uniform patch or single straight edge is ambiguous. The predicted point is $p_1 = p_0 + d$. For larger motion, estimate at coarse resolution, double the displacement when moving one pyramid level finer, and refine by warping and solving for a correction [1]. This is local translation tracking; it does not remove occlusion or independently moving objects within the same window.

Bilinear interpolation

At a subpixel position $x = m + \alpha$, $y = n + \beta$, first interpolate horizontally: $a = (1 - \alpha)I_{m,n} + \alpha I_{m+1,n}$ and $b = (1 - \alpha)I_{m,n+1} + \alpha I_{m+1,n+1}$. Interpolating vertically gives

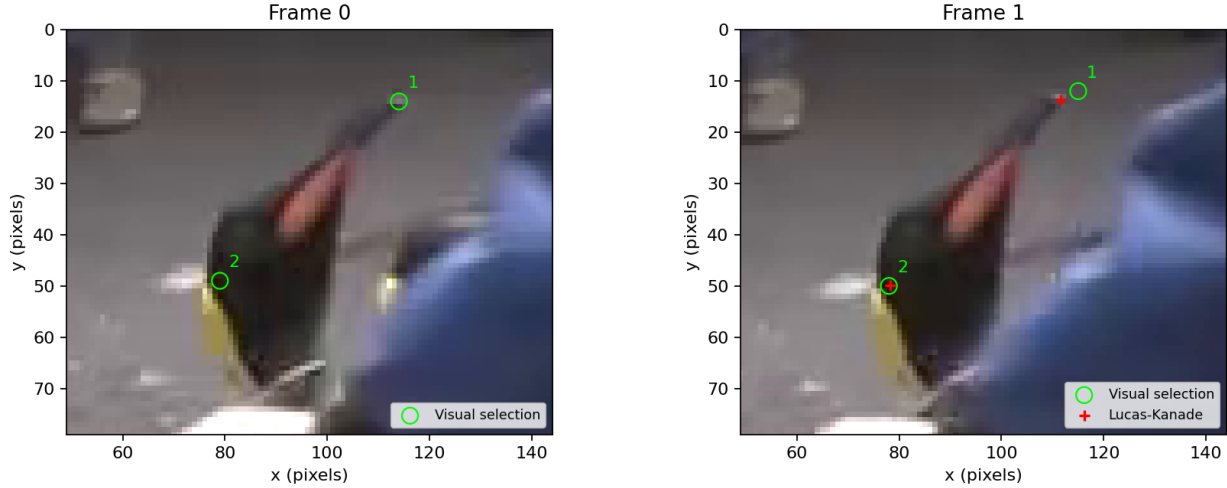
$$I(x, y) = (1 - \beta)a + \beta b = (1 - \alpha)(1 - \beta)I_{m,n} + \alpha(1 - \beta)I_{m+1,n} + (1 - \alpha)\beta I_{m,n+1} + \alpha\beta I_{m+1,n+1}.$$

For $[10, 20; 30, 40]$, $\alpha = 0.25$ and $\beta = 0.5$, the row values are 12.5 and 32.5, giving 22.5. The weights sum to one. Interpolation permits evaluating the warped image at non-integer coordinates during iterative tracking.

Pixel validation: penguins

The first two decoded frames are at source times 30.00 and 30.04 seconds. Pixel coordinates use the resized frames, origin at the upper left, x rightward and y downward. Corresponding points were visually selected from enlarged images independently of the tracker, rounded to integer pixels, and stored in `measurements.csv`. These are approximate annotations (roughly 1–2 pixels), not exact ground truth. The same 21×21 window and pyramid settings are used for validation and the flow video. The error is $e = \sqrt{(\hat{x}_1 - x_1)^2 + (\hat{y}_1 - y_1)^2}$.

Point	Initial p_0	Measured p_1	LK prediction	Error (px)
Beak tip	(114,14)	(115,12)	(111.62,13.75)	3.81
Neck patch	(79,49)	(78,50)	(78.23,49.96)	0.23



Numerical normal equations. For the neck point, use 441 pixels centered at $(79, 49)$, unnormalized grayscale values, 3×3 Sobel derivatives divided by 8, and $I_t = I_1 - I_0$. The computed system is

$$\begin{bmatrix} 125548.656 & -20539.938 \\ -20539.938 & 44011.094 \end{bmatrix} d = \begin{bmatrix} -118707.625 \\ 58378.125 \end{bmatrix}, \quad d = \begin{bmatrix} -0.7887 \\ 0.9583 \end{bmatrix}.$$

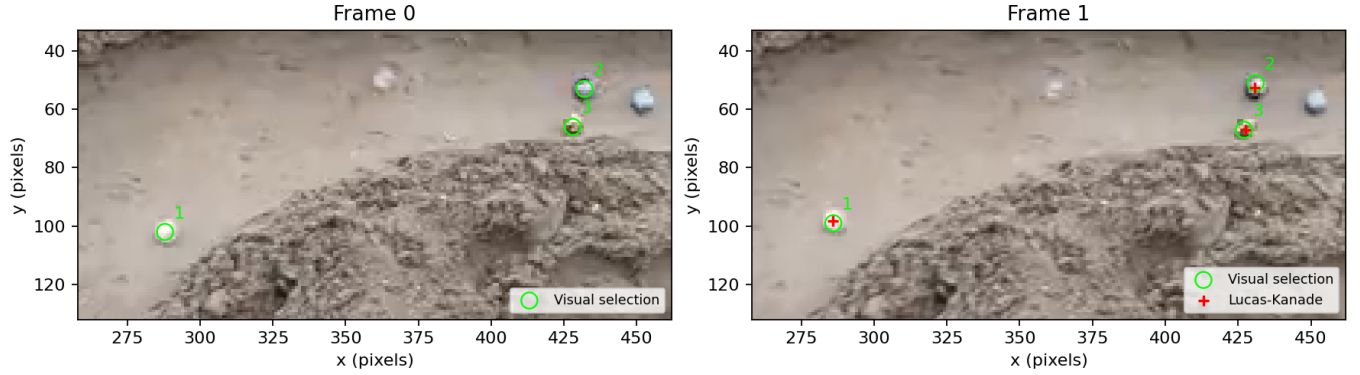
The single linear step predicts $(78.2113, 49.9583)$, approximately 0.22 pixels from the visual annotation. Pyramidal iterative LK predicts $(78.2311, 49.9583)$; these need not be identical because one linearization is not the full iterative method. All systems, including the beak failure, are exported in `tracking_calculations.json`.

Evidence and interpretation. The neck point moves approximately left and down by $(-0.77, 0.96)$ pixels/frame, or $(-19.2, 24.0)$ pixels/second in image coordinates. The beak discrepancy is larger than the neck discrepancy and illustrates sensitivity to blur, weak localization, and mixed motion in a patch. The enlarged frames show a nearby moving sleeve; this is not a controlled static-camera scene. These are apparent image motions, not physical walking speeds.

Pixel validation: marble race

The first pair is at source times 120.00 and 120.02 seconds. The same coordinate system, independent visual annotation procedure, and tracker settings are used. Numbers 1, 2 and 3 in the figure identify the white, blue, and yellow marbles.

Point	Initial p_0	Measured p_1	LK prediction	Error (px)
White marble	(288,102)	(286,99)	(285.96,98.23)	0.77
Blue marble	(432,53)	(431,51)	(430.77,52.69)	1.71
Yellow marble	(428,66)	(427,67)	(427.60,66.99)	0.60



Numerical normal equations. At the white marble, the same full-resolution linearization gives

$$\begin{bmatrix} 14677.734 & 983.203 \\ 983.203 & 26255.609 \end{bmatrix} d = \begin{bmatrix} -4397.375 \\ -22360.625 \end{bmatrix}, \quad d = \begin{bmatrix} -0.2432 \\ -0.8425 \end{bmatrix}.$$

This predicts (287.7568, 101.1575), approximately 2.78 pixels from the annotation. The full iterative pyramid improves this example to 0.77 pixels. A single small-motion linearization is inadequate for this patch’s displacement.

Evidence and interpretation. The white marble’s estimated displacement is $(-2.04, -3.77)$ pixels/frame, while the yellow marble’s is $(-0.40, 0.99)$. They have different apparent directions in this pair. The white marble’s image speed is approximately $50\sqrt{2.04^2 + 3.77^2} = 214$ pixels/second. This cannot be converted to centimeters/second without scene scale and camera geometry. The annotations and results support local image-motion estimates, but the blue marble’s 1.71-pixel error is substantial relative to its few-pixel diameter.

The race footage includes camera motion; background flow is therefore not evidence that the sand moves. A 21×21 patch around a tiny marble contains much more background than marble, which can bias the estimate. Sparse corner detection can also select sand texture or the on-screen scoreboard. Thus the flow videos do not establish persistent marble identities, race ranking, or reliable object speeds. The reported errors are examples, not an accuracy estimate over the whole video.

Reproducing Part A

Run the two commands in the README for `validate.py`. Each produces a 30-second flow video, CSV of flow vectors, two original frames, a flow overlay, validation figure/table, and numerical normal-equation systems. The web app runs the same code for its bundled samples. Uploaded videos have no predefined manual annotations and therefore do not receive a claimed validation score.

B. Four-view planar structure from motion

Data provenance and assumptions. This is a constructed, simulated example, not photographs or a reconstruction of either moving-video subject. The object is a flat L-shaped card. Its six boundary points, in centimeters, are

$$S_{\text{reference}} = \begin{bmatrix} -2 & 2 & 2 & 0 & 0 & -2 \\ -1 & -1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

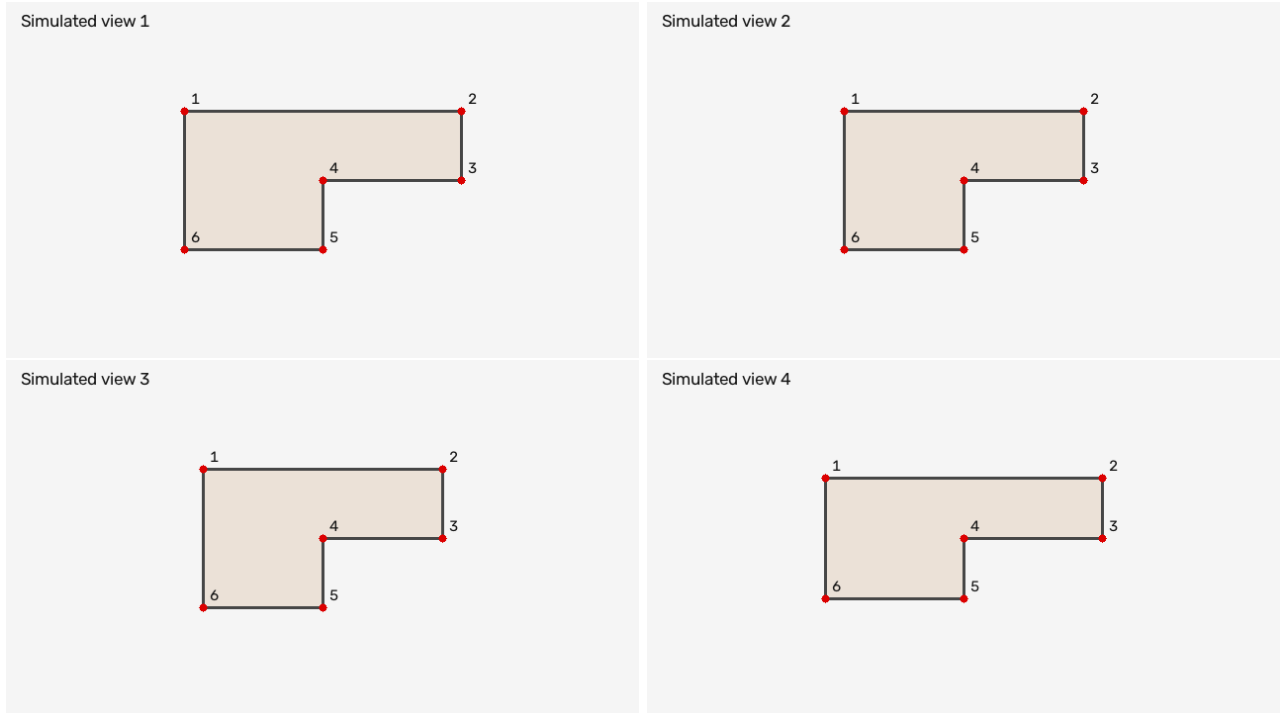
Four virtual orthographic cameras render images of this stationary object. The scale is 70 pixels/cm, image size is 640×360 , and image offset is $(320, 180)$. There is no perspective focal length, distortion, or perspective calibration matrix in this camera model. The first view is known to face the plane directly. These assumptions explicitly supply the metric information that a planar uncalibrated factorization cannot determine by itself [2].

Let $R_f = R_x(\text{pitch})R_y(\text{yaw})$, with

$$R_y(a) = \begin{bmatrix} \cos a & 0 & \sin a \\ 0 & 1 & 0 \\ -\sin a & 0 & \cos a \end{bmatrix}, \quad R_x(b) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos b & -\sin b \\ 0 & \sin b & \cos b \end{bmatrix}.$$

The first two rows are the image axes. Put the camera center at $C_f = 10R_f^T(0, 0, 1)^T$ cm, looking along its negative third axis. The projection is $p_{f,j} = (320, 180)^T + 70[R_f]_{1:2,:}(P_j - C_f)$; coordinates are rounded to image pixels. Positions below are simulation inputs, not estimated distances.

View	Yaw	Pitch	Camera center (cm)
1	0°	0°	(0,0,10)
2	30°	0°	(-5,0,8.6603)
3	-30°	0°	(5,0,8.6603)
4	0°	30°	(0,5,8.6603)



Views 2 and 3 have the same projections for this plane under orthography, despite opposite camera positions. This is a real ambiguity of the selected model. The JSON output provides all four images' coordinates, rotations and camera centers.

Centering, rank and reconstruction

Subtract each image's centroid (320, 180) and divide by 70. Stack the horizontal and vertical coordinates for each view to make the 8×6 observation matrix $W = MS$ [2]. The plane is centered and has rank 2, so a full rank-3 metric upgrade would be singular. The measured matrix in centimeters is

$$W = \begin{bmatrix} -2 & 2 & 2 & 0 & 0 & -2 \\ -1 & -1 & 0 & 0 & 1 & 1 \\ -1.7286 & 1.7286 & 1.7286 & 0 & 0 & -1.7286 \\ -1 & -1 & 0 & 0 & 1 & 1 \\ -1.7286 & 1.7286 & 1.7286 & 0 & 0 & -1.7286 \\ -1 & -1 & 0 & 0 & 1 & 1 \\ -2 & 2 & 2 & 0 & 0 & -2 \\ -0.8714 & -0.8714 & 0 & 0 & 0.8714 & 0.8714 \end{bmatrix}.$$

Factor $W = U\Sigma V^T$ and retain the two largest singular values. Let $\widehat{M} = U_2\Sigma_2^{1/2}$ and $\widehat{S} = \Sigma_2^{1/2}V_2^T$. The remaining ambiguity is $M = \widehat{M}Q$, $S = Q^{-1}\widehat{S}$. Our *known* first camera has in-plane rows equal to the identity, so

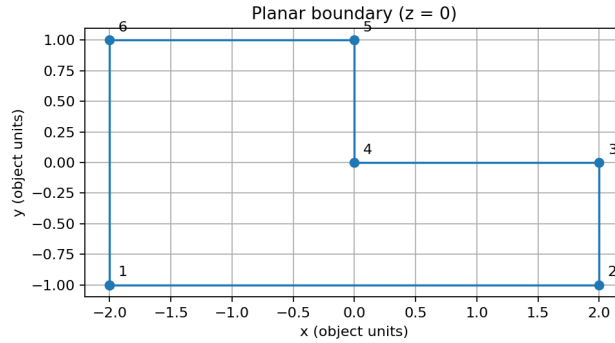
$$Q = \left(\widehat{M}_{1:2,:}\right)^{-1}, \quad M = \widehat{M}Q, \quad S = Q^{-1}\widehat{S}.$$

This is a calibrated planar specialization of the slide's factorization method; it does not claim to recover unknown general camera poses or depth. In general 3D, the slides determine the metric upgrade from unit-length, mutually perpendicular camera rows. Here their missing out-of-plane components and the planar rank loss make a direct full rank-3 implementation inappropriate.

The recovered motion rows are approximately

$$M = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0.8643 & 0 \\ 0 & 1 \\ 0.8643 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0.8714 \end{bmatrix}.$$

For example, point 1 in view 2 is $(320, 180) + 70(-1.7286, -1) = (199, 110)$ pixels. Recovering the six planar points gives $(-2, -1), (2, -1), (2, 0), (0, 0), (0, 1), (-2, 1)$, all with $z = 0$ by the planar assumption. Connecting them in their supplied boundary order estimates the L-shaped outline.



The default's reprojection RMS is below 10^{-10} pixels. This near-zero value reflects internally consistent simulated data, not real-camera accuracy. The first front-parallel image already fixes the planar shape and scale; the other views supply multi-view consistency and in-plane camera projections, not new depth.

Calculations, limitations and capture replacement

The program exports the centroids, full W , singular values, metric transform Q , motion matrix, reconstructed coordinates, and reprojection RMS in `sfm_calculations.json`. The RMS is

$$\text{RMS} = \sqrt{\frac{1}{4N} \sum_{f=1}^4 \sum_{j=1}^N \|p_{f,j} - [70(MS)_{f,j} + \bar{p}_f]\|^2}.$$

Camera translations along the viewing direction are not observable under orthographic projection. The distances in the camera table are supplied simulation settings. Reflection ambiguity remains in the tilted views of a perfectly flat object. The first view and known scale resolve the in-plane affine/scale ambiguity; the assumed plane resolves z , rather than estimating nonzero depth from these data.

For a photographed version, use a stationary flat object with labeled corners and four views at similar distance and fixed zoom. Capture a front-parallel first view, measure a known edge to set pixels/cm, and record camera settings and camera positions. Keep perspective effects small for the orthographic approximation. Supply the same ordered boundary coordinates in each image through `sfm.py --input data.json --out output/structure`. Actual images and measured camera metadata would still need to be supplied if the instructor requires captured rather than simulated data; this report does not represent the simulation as such a capture. Reprojection error then measures agreement with the image coordinates and should not be interpreted as independent 3D ground truth.

References

1. Shree K. Nayar. *Optical Flow*, FPCV-4-3, April 2025. Course-provided lecture; brightness constancy, least-squares Lucas–Kanade, and coarse-to-fine estimation, printed pp. 5–13.
2. Shree K. Nayar. *Structure from Motion*, FPCV-4-4, April 2025. Course-provided lecture; orthographic projection, centering, rank and factorization, printed pp. 2–15.
3. Shree K. Nayar. *Object Tracking*, FPCV-5-1. Course-provided lecture; tracking context. The implementation here uses local optical flow, not an object identity model.
4. Penguin Walk – Zoo Basel [HD]. Source video: <https://www.youtube.com/watch?v=fdk7X4LrgS4>. User-provided copy; sample 00:30–01:00.
5. SAND MARBLE RACE, S7: Round 1 – Jelle’s Marble Runs. <https://www.youtube.com/watch?v=0sHu99vdz-A>. User-provided copy; sample 02:00–02:30.